

Extensión de la Práctica Principal de Procesadores de Lenguajes (Examen febrero 2022)

Esta práctica consiste en la implementación mediante JFlex y Cup de un compilador de un pequeño lenguaje de programación, similar a C, denominado lenguaje PLX. El lenguaje PLX-2022 es una extensión del lenguaje PLX que se describe como prácticas de la asignatura, pero en esta versión extendida se requieren algunas funciones adicionales. Se presupone que todos los elementos del lenguaje PLX anteriores están presentes en el lenguaje PLX-2022 y que no se modifica su funcionamiento al incluir los nuevos elementos de esta extensión, aunque obviamente en este examen solo se evaluarán los casos que se describen en este enunciado, minimizando en la medida de lo posible el uso de funcionalidades anteriores. En concreto, para este ejercicio se utilizarán solo el tipo “int” y el constructor de tipos “set”.

EL CÓDIGO FUENTE (Lenguaje PLX):

El lenguaje PLX-2022 incluye todas las sentencias de control definidas en el lenguaje PLX anterior y alguna más. El lenguaje intermedio CTD, no es necesario modificarlo.

ASPECTOS LEXICOS

Cualquier duda que surja al implementar, y que no estuviera suficientemente clara en este enunciado debe resolverse de acuerdo a las especificaciones del lenguaje JAVA o bien el lenguaje C .

COMPILADOR DE PRUEBA

Se proporciona una versión compilada del compilador, que puede usarse como referencia para la generación de código. No es necesario que el código generado por el compilador del alumno, sea exactamente igual al generado por el compilador de prueba, basta con que produzca los mismos resultados al ejecutarse para todas las entradas.

El compilador de prueba se entrega solamente a título orientativo. Si hubiese errores en el compilador de prueba, prevalecen las especificaciones escritas en este enunciado. Estos posibles errores en ningún caso eximen al alumno de realizar una implementación correcta.

NOTA:

La versión compilada de “plx” se ha actualizado en el repositorio para cubrir los cambios requeridos en este ejercicio, por lo que aunque ya se hubiera descargado anteriormente debería actualizarse.

Conjuntos de números enteros.

Se añade el tipo conjunto, similar al tipo vector o array unidimensional, y compatible con este tipo hacia atrás, es decir, un array unidimensional se puede convertir implícitamente en un conjunto, pero no al revés. (NOTA: La implementación interna de los conjuntos coincide con la del array unidimensional)

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>set int c; c = {1,2,3}; print(c);</pre>		1 2 3
<pre>set int c; c = {1,2,3}; c = {4,5,6}; print(c);</pre>		4 5 6
<pre>set int c; c = {1,2,3}; set int d; d = {4,5,6}; print(c); print(d);</pre>		1 2 3 4 5 6
<pre>set int c; c = {1,2,3}; set int d; d = c; print(d);</pre>		1 2 3
<pre>set int a,b,c; a = {2,3}; b = {3,5,7}; c = a; print(c); print(b); b = c; a = b; print(a); print(c);</pre>		2 3 3 5 7 2 3 2 3
<pre>set int c; c = {1,2,3}; set int d; c = c; d = c; print(c); print(d);</pre>		1 2 3 1 2 3
<pre>int a[3]; set int c; a[0] = 1; a[1] = 2; a[2] = 3; c = a; print(c);</pre>		1 2 3
<pre>int a[4]; set int c; a[0] = 1; a[1] = 2; a[2] = 3; c = a; print(c);</pre>		1 2 3 0
<pre>int a[3]; set int c; a[0] = 1; a[2] = 3; c = a; print(c);</pre>		1 0 3

IMPORTANTE: Al imprimir los elementos del conjunto da igual el orden en el que se muestren.

Inicialización de conjuntos

Al igual que otras variables en PLX, la declaración de la variable puede incluir también una inicialización

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>set int c = {1,2,3}; print(c);</pre>		1 2 3
<pre>set int c = {1,2,3}, d = {4,5,6}; print(c); print(d);</pre>		1 2 3 4 5 6
<pre>set int c = {1,2,3}; set int d = c; print(c); print(d);</pre>		1 2 3 1 2 3
<pre>set int c = {1,2,3,4}, d = c; print(c); print(d);</pre>		1 2 3 4 1 2 3 4

Unicidad de elementos.

A diferencia de los arrays, el número de elementos de un conjunto es variable y puede incrementarse durante la ejecución del programa. Otra diferencia sustancial es que en un conjunto no puede haber elementos repetidos, y en un array sí.

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>set int c; c = {1,2,2,3,3,3}; print(c);</pre>		1 2 3
<pre>set int c = {1,2,2,3,3,3}; print(c);</pre>		1 2 3
<pre>set int c = {1,2,3,1,3,5}; set int d = c; print(d);</pre>		1 2 3 5
<pre>int a[3]; set int c; a[0] = 2; a[1] = 2; a[2] = 2; c = a; print(c);</pre>		2
<pre>int a[6] = {1,1,3,3,4,5}; set int c = a; print(c);</pre>		1 3 4 5

Compatibilidad con arrays.

Un array unidimensional de enteros se puede convertir en un conjunto, (como se ha hecho en los ejercicios anteriores) pero no a la inversa. Tampoco se puede acceder a los elementos del conjunto de forma indexada como se hace con los arrays.

Los siguientes ejemplos funcionarían si la variable `c` se hubiera definido como array unidimensional

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>set int c; c = {1,2,3}; print(c[0]); print(c[2]);</pre>	<pre>... error; # ERROR: La variable no es de tipo array</pre>	
<pre>int a[2]; set int c = {1,2}; a[0] = c[0]; a[1] = c[1]; print(a);</pre>	<pre>... error; # ERROR: La variable no es de tipo array</pre>	
<pre>int a[3]; set int c; c = {1,2,3}; a = c; print(c);</pre>	<pre>... error; # No se puede asignar un conjunto a un array</pre>	

Operador size.

Devuelve el número de elementos del conjunto.

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>set int c; c = {1, 2, 3}; print(c.size);</pre>		3
<pre>set int c; c = {1, 2, 3, 2, 1}; print(c.size);</pre>		3
<pre>set int c; print(c.size);</pre>		0

Sentencias condicionales con conjuntos.

Se introduce una nueva condición booleana para comprobar si un elemento pertenece o no a un conjunto.

Esta condición puede usarse en cualquier sentencia de control o en expresiones booleanas

Código fuente (PLX)	Código intermedio (CTD)	Resultado de la ejecución
<pre>set int c; c = {1,2,3}; if (3 in c) { print(3); } if (0 in c) { print(0); } print(4);</pre>		3 4
<pre>set int c; set int d; c = {1,2,3}; d = {2,3,4}; int i; for(i=1;i<5;i++) { if (i in c && i in d) { print(i); } }</pre>		2 3
<pre>set int c; set int d; c = {4,6,8,10}; d = {2,3,4}; int i,j; while(!(i in c i in d)) { i = i+1; j = i; while(j*2 in c) { print(j*2); j = j*2; } }</pre>		4 8

EL CÓDIGO OBJETO (Código de tres direcciones):

El código objeto CTD implementa una máquina abstracta con infinitos registros a los que se accede mediante variables. Todas las variables se consideran que están previamente definidas y que su valor inicial es 0. No se diferencia entre números enteros y reales, salvo para realizar operaciones.

El conjunto de instrucciones del código intermedio, y su semántica son las siguientes:

Instrucción	Acción
<code>x = a ;</code>	Asigna el valor de a en la variable x
<code>x = a + b ;</code>	Suma los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a - b ;</code>	Resta los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a * b ;</code>	Multiplica los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a / b ;</code>	Divide (div. entera) los valores de a y b, y el resultado lo asigna a la variable x
<code>x = a +r b ;</code>	Suma de dos valores reales
<code>x = a -r b ;</code>	Resta de dos valores reales
<code>x = a *r b ;</code>	Multiplicación de dos valores reales
<code>x = a /r b ;</code>	Division de dos valores reales
<code>x = (int) a ;</code>	Convierte un valor real a, en un valor entero, asignándose a la variable x
<code>x = (float) a ;</code>	Convierte un valor entero a, en un valor real, asignándose a la variable x
<code>x = y[a] ;</code>	Obtiene el a-esimo valor del array y, asignando el contenido en x
<code>x[a] = b ;</code>	Coloca el valor b en la a-esima posición del array x
<code>x = *y ;</code>	Asigna a x el valor contenido en la memoria referenciada por y.
<code>*x = y ;</code>	Asigna el valor y en la posición de memoria referenciada por x.
<code>x = &y ;</code>	Asigna a x la dirección de memoria en donde esta situado el obteto y.
<code>goto l ;</code>	Salto incondicional a la posición marcada con la sentencia "label l"
<code>if (a == b) goto l ;</code>	Salta a la posición marcada con la sentencia "label l", si y solo si el valor de a es igual que el valor de b
<code>if (a != b) goto l ;</code>	Salta a la posición marcada con la sentencia "label l", si y solo si el valor de a es distinto que el valor de b
<code>if (a < b) goto l ;</code>	Salta a la posición marcada con la sentencia "label l", si y solo si el valor de a es estrictamente menor que el valor de b.
<code>l:</code>	Indica una posición de salto.
<code>label l ;</code>	Indica una posición de salto. Es otra forma sintáctica equivalente a la anterior.
<code>function f :</code>	Indica una posición de salto para comienzo de una función.
<code>end f ;</code>	Indica el final del código de una función.
<code>param n = x;</code>	Indica que x debe usarse como parámetro n-simo en la llamada a la próxima función.
<code>x = param n ;</code>	Asigna a la variable x el valor del parámetro n-simo definido antes de la llamada a la función.
<code>call f ;</code>	Salto incondicional al comienzo de la función f. Al alcanzar la sentencia return el control vuelve a la instrucción inmediatamente siguiente a esta
<code>gosub l ;</code>	Salto incondicional a la etiqueta f. Al alcanzar la sentencia return el control vuelve a la instrucción inmediatamente siguiente a esta
<code>return ;</code>	Salta a la posición inmediatamente siguiente a la de la instrucción que hizo la llamada (call f) o (gosub l)
<code>write a ;</code>	Imprime el valor de a (ya sea entero o real)
<code>writec a ;</code>	Imprime el carácter Unicode correspondiente al número a
<code>print a ;</code>	Imprime el valor de a, y un salto de línea
<code>printc a ;</code>	Imprime el carácter Unicode correspondiente al número a, y un salto de línea
<code>error ;</code>	Indica una situación de error, pero no detiene la ejecución.
<code>halt ;</code>	Detiene la ejecución. Si no aparece esta instrucción la ejecución se detiene cuando se alcanza la última instrucción de la lista.
<code># ...</code>	Cualquier línea que comience con # se considera un comentario.
<code>. <nombre fichero></code>	Incluye el contenido del fichero indicado, buscándolo en el directorio actual.

En donde a,b representan tanto variables como constantes enteras, x,y representan siempre una variable, n representa un numero entero, l representa una etiqueta de salto y f un nombre de función.

IMPLEMENTACIÓN DE LA PRÁCTICA:

Se proporciona una solución compilada del ejercicio (versiones para Linux, Windows y Mac). Esto puede servir de ayuda para comprobar los casos de prueba y las instrucciones [de código](#) intermedio. Para compilar y ejecutar un programa en lenguaje PLX, pueden utilizarse las instrucciones

	Linux
Compilación	<code>./plx prog.plx prog.ctd</code>
Ejecución	<code>./ctd prog.ctd</code>

El programa a enviar para su corrección automática debe probarse previamente y comparar los resultados de la ejecución anterior. No es necesario que el código generado sea idéntico al que se propone como ejemplo (que de hecho no es óptimo), basta con que sea equivalente, es decir que dé los mismos resultados al ejecutar los casos de prueba.

	Linux
Compilación	<code>java PLXC prog.plx prog.ctd</code>
Ejecución	<code>./ctd prog.ctd</code>

En donde `prog.plx` contiene el código fuente en PLX, `prog.ctd` es un fichero de texto que contiene el código intermedio válido según las reglas gramaticales de este lenguaje. El programa `plx` es un *script* del *shell* del sistema operativo que llama a (`java PLXC`), que es el programa que se pide construir en este ejercicio. El programa `ctd` es un intérprete del código intermedio. Asimismo, para mayor comodidad se proporciona otro *script del shell* denominado `plx` que compila y ejecuta en un solo paso, y al que se pasa el nombre del fichero sin extensión.

	Linux
Compilación + Ejecución	<code>./plx prog</code>

NOTAS IMPORTANTES:

1. Toda práctica debe contener al menos tres ficheros denominados “`PLXC.java`”, “`PLXC.flex`” y “`PLXC.cup`”, correspondientes respectivamente al programa principal y a las especificaciones en JFlex y Cup. Para realizar la compilación se utilizarán las siguientes instrucciones:

```
cup PLXC.cup
jflex PLXC.flex
javac *.java
```

y para compilar y ejecutar el programa en PLX

```
java PLXC prog.plx prog.ctd
./ctd prog.ctd
```

2. Puede ocurrir que al descargar los ficheros y descomprimirlos en Linux se haya perdido el carácter de fichero ejecutable. Para poder ejecutarlos debe modificar los permisos:

```
chmod +x plx plxc ctd
chmod +x plx-linux plxc-linux ctd-linux
```

3. El programa `plx`, que implementa el compilador de `plx` y que sirve para comparar los resultados obtenidos, incluye una opción `+c` para generar comentarios que pueden ayudar a identificar el código generado para cada línea del programa fuente:

```
./plx +c prog.ctd
```

4. El programa `ctd`, interprete del código intermedio, tiene una opción `+v` que sirve para mostrar la ejecución del código de tres direcciones paso a paso y que pueden ayudar en la depuración de errores:

```
./ctd +v prog.ctd
```

5. Para realizar la corrección automática es necesario que la salida sea la misma que la esperada, independientemente del orden de los elementos del conjunto. Para esto se usa un pequeño script `lineSort.py` en Python

```
./ctd prog.ctd | python3 lineSort.py
```

6. Los ejemplos que se proponen como casos de prueba no definen exhaustivamente el lenguaje. Para implementar esta práctica es necesario generar otros casos de prueba de manera que se garantice un funcionamiento en todos los casos posibles, y no solo en este limitado banco de pruebas.

7. En todas las pruebas en donde el código `plx` produce un “*error*”, para comprobar que el compilador realmente detecta el error, se probará también que el código corregido compila adecuadamente, y si no es así la prueba no se considerará correcta.